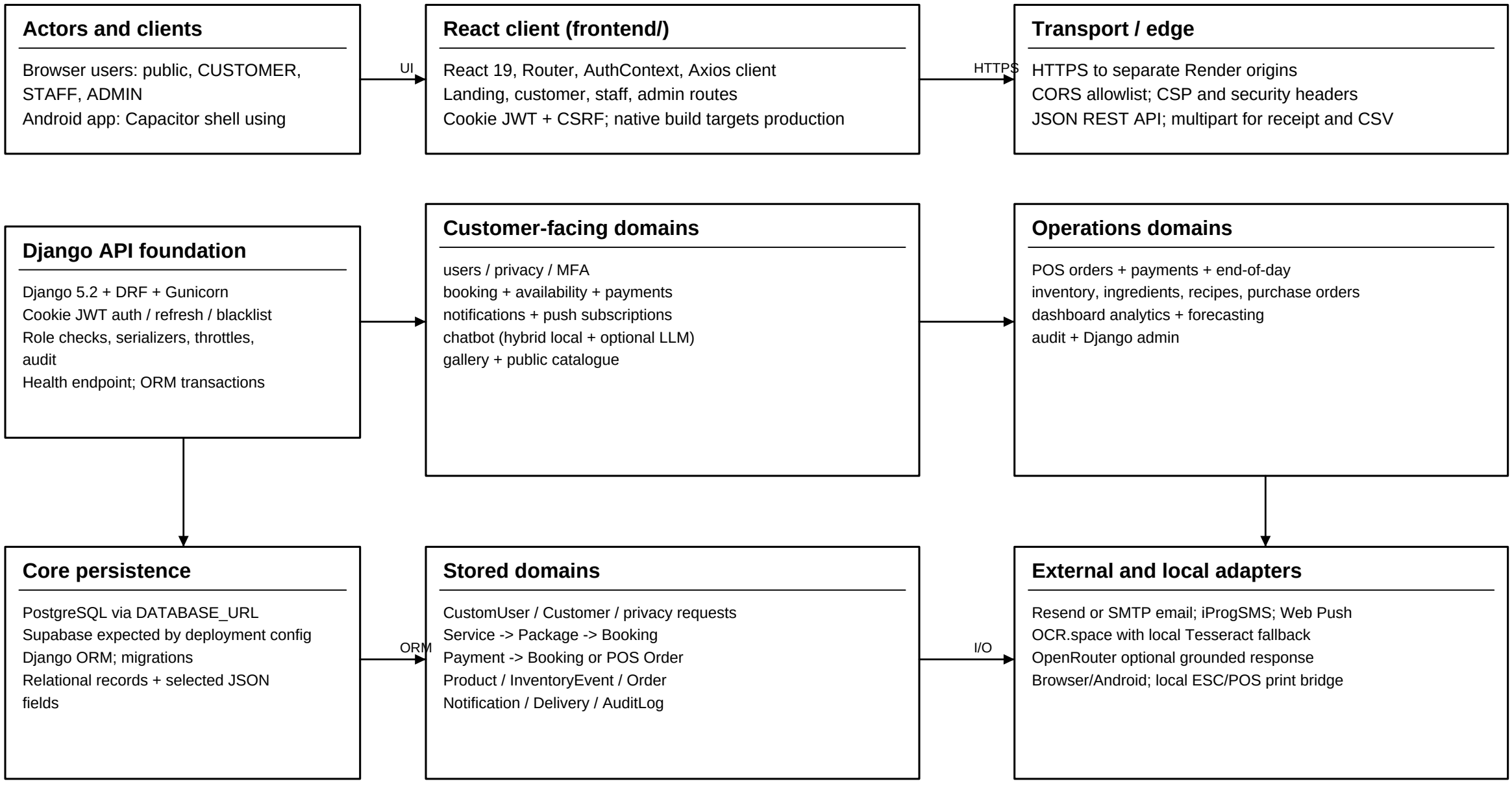
System Architecture

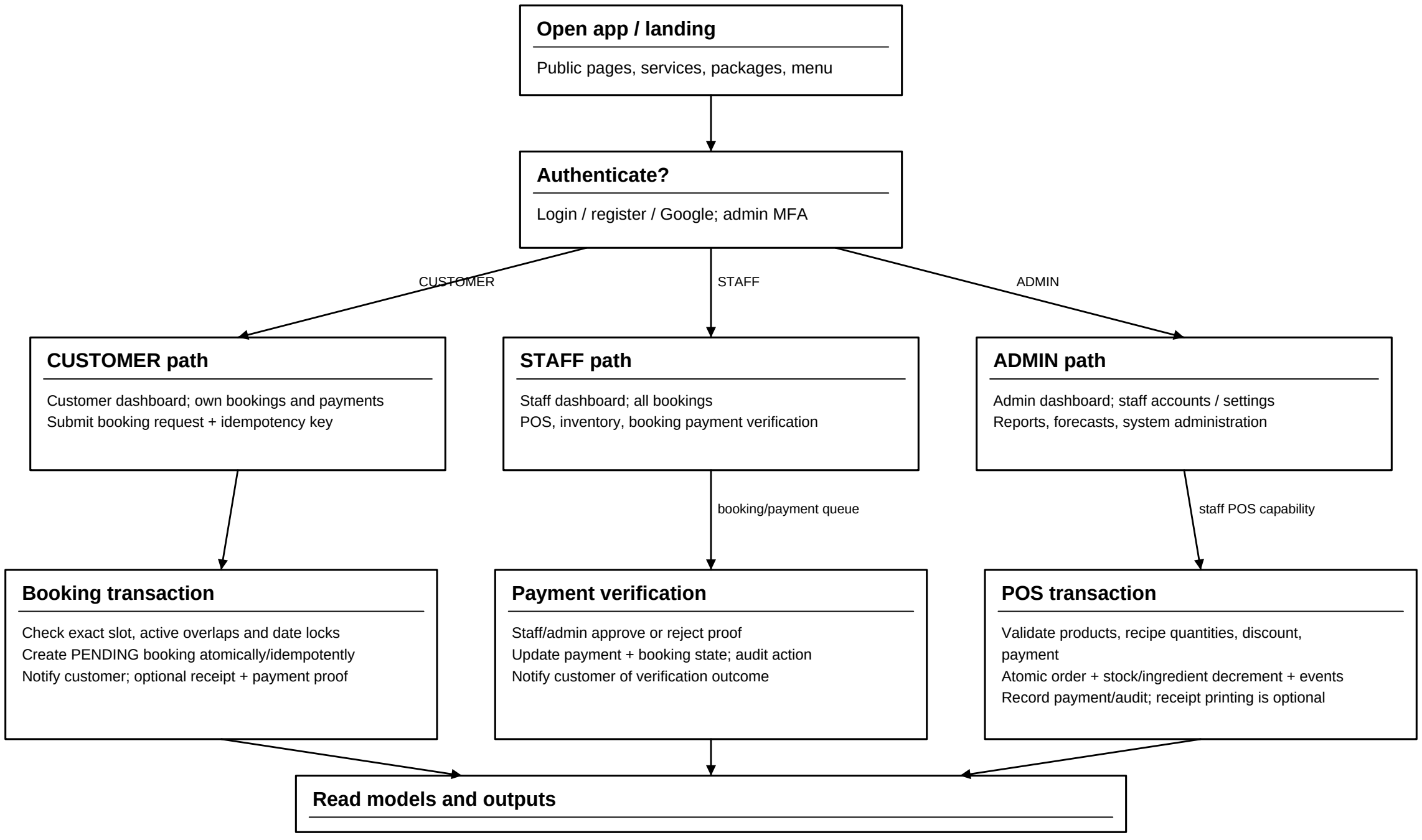
Runtime components and domain boundaries in the inspected source tree.



Sources: frontend/src/App.js; backend/config/{urls,settings}.py; backend/apps/*; backend/ml/*; render.yaml

Application Flowchart

Entry, authorization, booking and POS paths traced through client routes and API views.



Sources: frontend/src/App.js; backend/config/urls.py; booking/views.py; pos/views.py; notifications/booking_events.py

Role-Based Access Control (RBAC)

Three application roles plus anonymous/public access. API permissions and per-record filters are shown.

Capability	Public / Anonymous	CUSTOMER	STAFF	ADMIN
Landing catalogue	Read services, packages, gallery, active products	Read	Read	Read
Account and profile	Login/register/reset; no private records	Own profile, consent, export, requests	Own profile; admin manages accounts	Manage staff/users; reset protected
Booking records	No create/read	Create; own bookings only; edit rules/time window	All bookings; update/validate	All bookings; admin-only studio date blocks
Booking payments	Read payment methods	Submit own proof; own status/receipt	Verify APPROVED/REJECTED; view all	Verify APPROVED/REJECTED; view all
POS and inventory	Public active menu only	No operational writes	Orders/payments; product/stock/ingredient/PO operations	Same operational capabilities; reports admin-only EOD
Analytics / forecasting	No access	No access	Dashboard + forecast; upload history CSV	Dashboard + forecast; upload history CSV
Admin-only actions	No access	No access	No access	Staff CRUD, FAQ management, system reset, EOD reports,

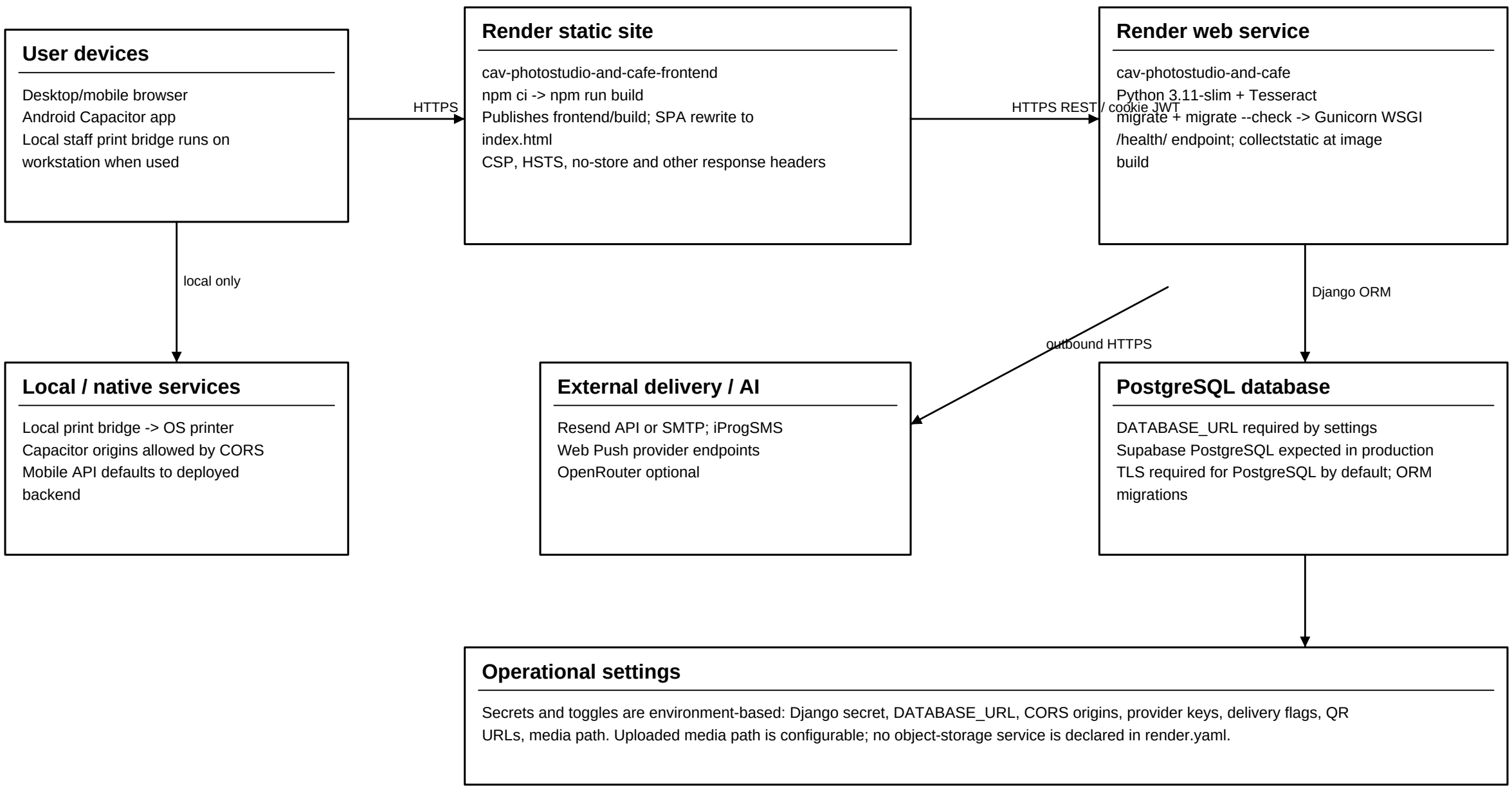
Enforcement notes

Default API access requires authentication; public endpoints explicitly opt out.
CUSTOMER booking/payment records are owner-filtered; STAFF and ADMIN share operations access.

Sources: users/permissions.py; users/views.py; booking/views.py; inventory/views.py; pos/views.py; dashboard/views.py; forecasting/views.py

Deployment Diagram

Production topology from Render configuration, Dockerfile, Django settings and Capacitor client configuration.



Sources: render.yaml; backend/Dockerfile; backend/config/settings.py; frontend/src/api/config.js; frontend/serve-build.js

Input - Process - Output (IPO)

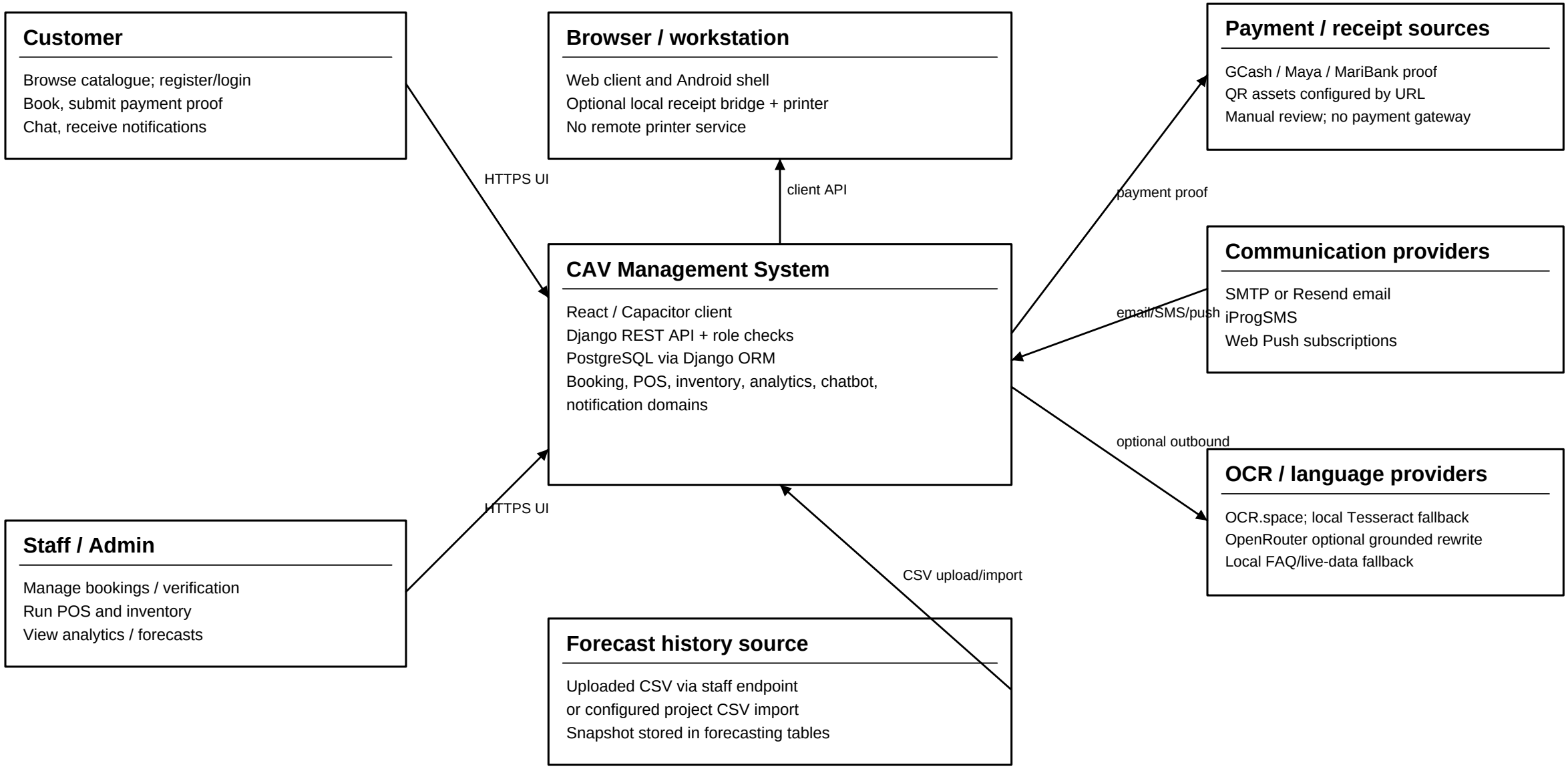
Inputs, processing rules and persisted/returned outputs for four implemented workflows.

INPUT	PROCESS	OUTPUT
BOOKING		
Authenticated customer: package, date, time, notes, idempotency key Payment proof / reference, if submitted	Check role, package, business hours, overlaps, unavailable dates Lock date and create/update transactionally; payment proof stored	Booking status + history; Payment linked to Booking Notification records; later staff verification changes status
POS SALE		
Staff cart with product IDs/qty, optional booking link, discount Payment details + optional print request	Validate active products and stock/recipe ingredients Atomic order/payment/stock updates; append InventoryEvent + AuditLog	Order with line_items and payment status Inventory counts/movements; printable receipt response
FORECAST		
Staff/admin CSV upload (<=10 MB) or configured source CSV Date, quantity/amount/items columns	Normalize aliases; parse rows; skip invalid; hash/dedupe; atomically activate snapshot Exclude cancelled/void/refund-like statuses; aggregate series; choose moving average or	Sales/demand forecast ranges, metrics, data-quality status Reorder recommendations using live inventory
CHATBOT / DASHBOARD		
Question/history/role OR date range/grain query Database bookings, orders, payments, inventory; approved knowledge	Safety checks; live lookup / FAQ retrieval; optional grounded OpenRouter Dashboard aggregation over paid orders, approved booking payments and inventory	Answer + source/citations; explicit action proposal/confirmation JSON metrics, trends, bookings, alerts, top products/packages

Sources: booking/views.py; booking/availability.py; pos/views.py; inventory/models.py; forecasting/*; chatbot/views.py; dashboard/views.py

System Context Diagram

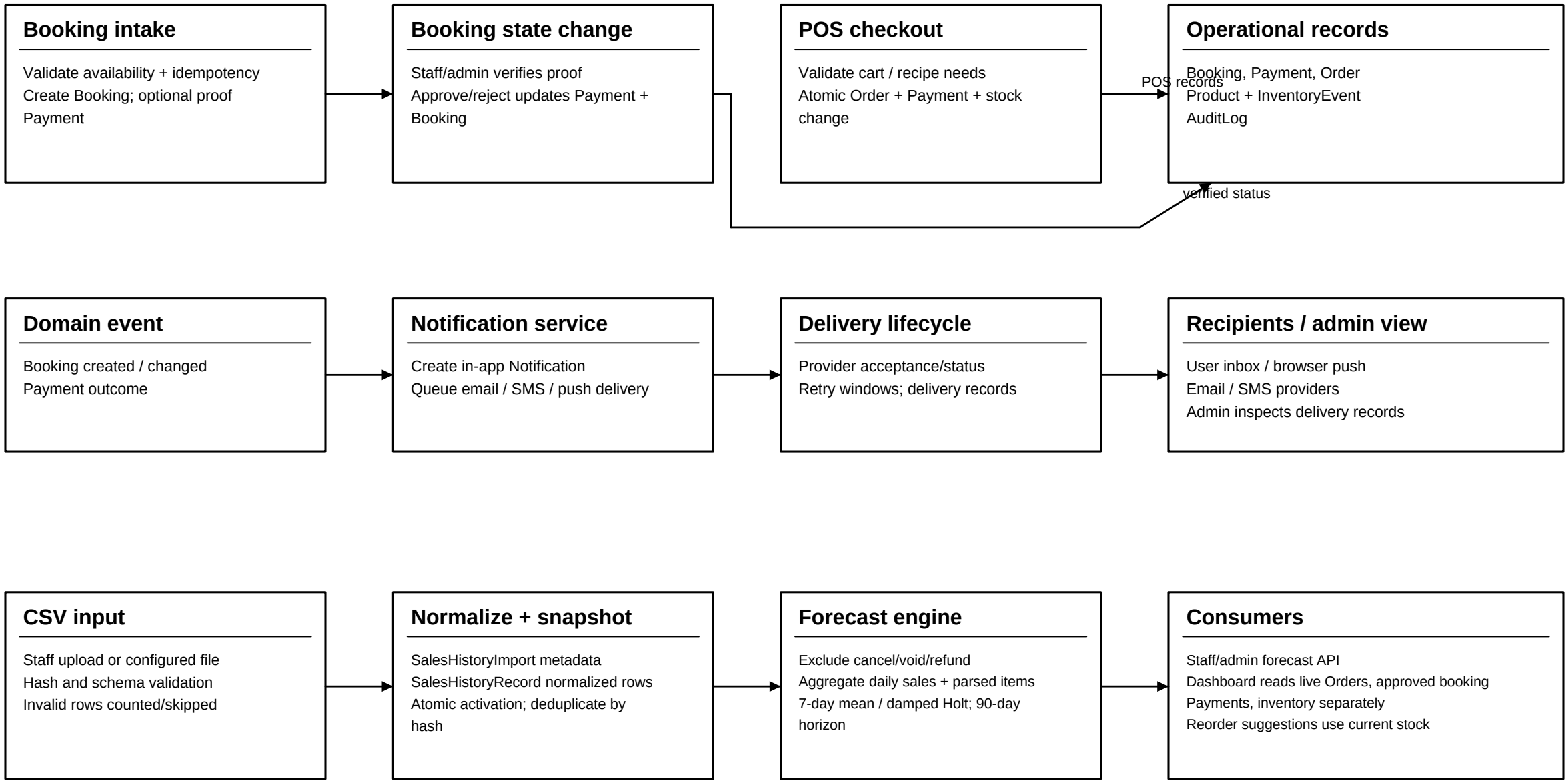
CAV system boundaries and the external actors/services that exchange data with the implemented application.



Sources: frontend/src/*; backend/config/urls.py; backend/config/settings.py; backend/apps/*; backend/ml/*

Data Pipeline

Persisted business events and analytical feeds; forecast-history ingestion is separate from operational POS transactions.



No POS-to-forecast ETL is present: imported CSV snapshots feed the forecast engine; operational records feed dashboard analytics.

Sources: pos/views.py; payment/models.py; inventory/models.py; booking/views.py; notifications/*; forecasting/*; dashboard/views.py